

build-adapter

Build Adapter

The build adapter provides an interface for build, test, and lint operations. It abstracts toolchain-specific commands behind a common contract, enabling challenges to validate that projects compile, pass tests, and satisfy linting rules.

BuildAdapter Interface

Defined in adapter_build.go:

```
type BuildAdapter interface {
    Build(ctx context.Context, target BuildTarget) (*BuildResult,
        error)
    RunTests(ctx context.Context, target TestTarget)
        (*TestResult, error)
    Lint(ctx context.Context, target LintTarget) (*LintResult,
        error)
    Available(ctx context.Context) bool
}
```

Method Summary

Method	Returns	Purpose
Build	*BuildResult	Execute a build target
RunTests	*TestResult	Execute a test suite
Lint	*LintResult	Execute a linting tool
Available	bool	Check if the build toolchain is installed

Target Types

BuildTarget

```
type BuildTarget struct {
    Name string `json:"name"`
}
```

```
    Task string `json:"task"`
    Args []string `json:"args"`
}
```

TestTarget

```
type TestTarget struct {
    Name string `json:"name"`
    Task string `json:"task"`
    Filter string `json:"filter"`
}
```

LintTarget

```
type LintTarget struct {
    Name string `json:"name"`
    Task string `json:"task"`
    Args []string `json:"args"`
}
```

Result Types

BuildResult

```
type BuildResult struct {
    Target string `json:"target"`
    Success bool `json:"success"`
    Duration time.Duration `json:"duration"`
    Output string `json:"output"`
    Artifacts []string `json:"artifacts"`
}
```

TestResult

```
type TestResult struct {
    Suites []TestSuite `json:"suites"`
    TotalTests int `json:"total_tests"`
    TotalFailed int `json:"total_failed"`
    TotalErrors int `json:"total_errors"`
    TotalSkipped int `json:"total_skipped"`
    Duration time.Duration `json:"duration"`
    Output string `json:"output"`
}
```

LintResult

```
type LintResult struct {  
    Tool      string    `json:"tool"`  
    Success   bool      `json:"success"`  
    Duration  time.Duration `json:"duration"`  
    Warnings  int       `json:"warnings"`  
    Errors    int       `json:"errors"`  
    Output    string    `json:"output"`  
}
```

Built-in Implementations

GradleCLIAdapter

For Gradle-based projects (Java, Kotlin, Android).

```
adapter := userflow.NewGradleCLIAdapter("/path/to/project", false)  
// Set useContainer=true to run via podman-compose
```

Operation	Command	Result Parsing
Build	./gradlew <task>	Exit code determines success
Test	./gradlew <task> [--tests <filter>]	JUnit XML from build/test-results/
Lint	./gradlew <task>	Exit code determines success
Available	Checks for gradlew in project root	--

When useContainer is true, commands are prefixed with podman-compose run --rm build.

Test result parsing searches for JUnit XML files in build/test-results/ and app/build/test-results/, parses them with ParseJUnitXML, and converts to TestResult via JUnitToTestResult.

NPMCLIAdapter

For Node.js projects (React, Vue, Angular, plain Node).

```
adapter := userflow.NewNPMCLIAdapter("/path/to/project")
```

Operation	Command	Result Parsing
Build	npm run <task>	

Operation	Command	Result Parsing
		Exit code determines success
Test	<code>npx vitest run --reporter=junit --outputFile=<tmp></code>	JUnit XML from temp file
Lint	<code>npx eslint . --format=json</code>	JSON output parsed for error/warning counts
Available	Checks for package.json in project root	--

Test results are produced by Vitest in JUnit format, written to a temporary file, and parsed. Lint results parse ESLint JSON output to extract per-file error and warning counts.

GoCLIAdapter

For Go projects.

```
adapter := userflow.NewGoCLIAdapter("/path/to/project")
```

Operation	Command	Result Parsing
Build	<code>go build <task or ./...></code>	Exit code determines success
Test	<code>go test -json <task or ./...> [-run <filter>]</code>	JSON event stream parsed
Lint	<code>go vet <task or ./...></code>	Non-empty output lines counted as errors
Available	Checks for go.mod in project root	--

Test result parsing reads the JSON lines stream from `go test -json`. Each line is a `goTestEvent` with fields: Time, Action, Package, Test, Elapsed, Output. Events with Action of pass, fail, or skip are counted per package and converted to `TestSuite` and `TestCase` structures.

CargoCLIAdapter

For Rust projects.

```
adapter := userflow.NewCargoCLIAdapter("/path/to/project")
```

Operation	Command	Result Parsing
Build	<code>cargo build</code>	Exit code determines success
Test	<code>cargo test [<filter>] -- --format=json -Z unstable-options</code>	JSON event stream parsed
Lint	<code>cargo clippy -- -D warnings</code>	Lines containing "warning:" or "error:" counted
Available	Checks for Cargo.toml in project root	--

Test result parsing reads JSON lines from Cargo's test output. Events with type: "test" and event of ok, failed, or ignored are counted.

JUnit XML Parsing

The package includes a JUnit XML parser (`ParseJUnitXML`) that handles both `<testsuites>` wrappers and standalone `<testsuite>` elements. It supports the standard JUnit XML attributes: name, tests, failures, errors, skipped, time, classname.

`JUnitToTestResult` converts parsed JUnit suites into the framework's `TestResult` type with aggregated counts.

Result Parsers

Helper functions in `result_parser.go` convert results to maps for assertion evaluation:

```
// For test results:
values := userflow.ParseTestResultToValues(testResult)
// Returns: total_tests, total_failed, total_errors,
//          total_skipped,
//          duration_ms, suite_count, output, all_tests_pass

metrics := userflow.ParseTestResultToMetrics(testResult)
// Returns MetricValue entries for total_tests, total_failed,
//          total_errors, total_skipped, duration

// For build results:
values := userflow.ParseBuildResultToValues(buildResult)
// Returns: target, success, duration_ms, output, artifact_count

metrics := userflow.ParseBuildResultToMetrics(buildResult)
```

```
// Returns MetricValue entries for build_success, build_duration,  
// artifact_count
```

Example: Multi-Target Build Challenge

```
adapter := userflow.NewGoCLIAdapter("/path/to/project")  
  
challenge := userflow.NewBuildChallenge(  
    "CH-BUILD-001",  
    "Build All Targets",  
    "Verify all build targets compile successfully",  
    nil,  
    adapter,  
    []userflow.BuildTarget{  
        {Name: "api-server", Task: "./cmd/server"},  
        {Name: "cli-tool", Task: "./cmd/cli"},  
    },  
)
```

Example: Test Suite Challenge

```
challenge := userflow.NewUnitTestChallenge(  
    "CH-TEST-001",  
    "Unit Tests",  
    "Run all unit tests and verify zero failures",  
    []challenge.ID{"CH-BUILD-001"},  
    adapter,  
    []userflow.TestTarget{  
        {Name: "all-packages", Task: "./..."},  
    },  
)
```

Example: Lint Challenge

```
challenge := userflow.NewLintChallenge(  
    "CH-LINT-001",  
    "Code Quality",  
    "Run linters and verify zero errors",  
    []challenge.ID{"CH-BUILD-001"},  
    adapter,  
    []userflow.LintTarget{  
        {Name: "go-vet", Task: "./..."},  
    },  
)
```